



Fine-tune Once, Serve Anywhere: Transformers, Unsloth, vLLM & Ollama

Proyecto Final

Mario Alonso Alonso

Rodrigo Covas Vasquez

Jorge Enebral Alonso

Pedro Meseguer Torregrosa

Procesamiento de Lenguaje Natural

4º Grado en Ingeniería Matemática e Inteligencia Artificial

Índice

1. Introducción y motivación.	3
2. Metodología	4
2.1. Metodología para fine-tuning.....	4
2.2. Metodología para inferencia	4
3. Evaluación.....	6
3.1. Métricas	6
3.2. Resultados cuantitativos.....	6
4. Comparación de frameworks	8
4.1. Comparación centrada en fine-tuning.....	8
4.2. Comparación centrada en inferencia.....	8
5. Discusión, conclusiones y trabajo futuro.....	10

1. Introducción y motivación.

Este proyecto final se centra en la experimentación y el análisis de técnicas de fine-tuning y despliegue de Large Language Models (LLMs) mediante cuatro frameworks actuales: Transformers, Unsloth, vLLM y Ollama. El objetivo es analizar cómo distintas decisiones de implementación afectan a la eficiencia de entrenamiento (tiempo y memoria) y al rendimiento de inferencia (latencia, throughput y consumo de recursos), además de la experiencia de desarrollo con estas herramientas.

La motivación principal es adquirir experiencia práctica con métodos de fine-tuning eficiente, como el PEFT (parameter-efficient fine-tuning) y la cuantización, para adaptar y servir modelos en escenarios con recursos limitados sin comprometer en exceso la calidad de las respuestas. Al analizar los distintos frameworks antes y después de los ajustes, se busca entender los posibles trade-offs que puedan tener, e identificar los casos de uso para cada una de las distintas herramientas. Dado que el enfoque del proyecto es un análisis reproducible y la comprensión del proceso, se priorizan modelos y datasets compactos frente a otros más grandes que nos permitirían obtener mejores resultados.

2. Metodología

2.1. Metodología para fine-tuning

En primer lugar, se ha hecho downsampling del dataset inicial para trabajar con solamente un 15% del mismo. Dado que la naturaleza de este proyecto es comparativa, no es fundamental obtener modelos altamente precisos, sino que se prioriza poder hacer más ejecuciones del entrenamiento. Al ser el entrenamiento más corto, obtenemos como consecuencia más libertad para hacer pruebas y llegar a mejores conclusiones.

Se ha hecho fine-tuning del modelo base proporcionado con Transformers + PEFT y con Unsloth, para así observar el comportamiento de dos métodos distintos y relacionarlo con sus características a priori. En términos de la implementación, Unsloth ha destacado por su extrema dificultad para ser ejecutado de forma local, puesto que los requisitos son realmente demandantes (GPU, Linux, versiones incompatibles, y otros errores varios). En consecuencia, se ha acudido forzosamente a la implementación de los desarrolladores en Google Colab para, partiendo de ese punto, ajustarla a las necesidades pertinentes.

Los parámetros empleados en el entrenamiento se basan principalmente en la configuración de serie de los modelos. Sin embargo, gracias a lo anteriormente mencionado, se han podido comparar varios valores, como el del learning rate, el parámetro r del rango de las matrices, el número de épocas, para finalmente seleccionar los que ofrecían resultados más satisfactorios. En cuanto a la técnica utilizada, LoRA se posiciona como la elección final, ya que es fiable, simple de emplear, y en consecuencia, la opción recomendada por los propios desarrolladores de los modelos.

Además, hemos decidido cuantizar el modelo a 4 bits, acelerando el fine-tuning y disminuyendo los recursos en memoria. Un QLoRA básicamente.

2.2. Metodología para inferencia

En primer lugar, la inferencia se podría realizar con el modelo base o con el modelo fine-tuneado. En cuanto a las respuestas, conviene usar el modelo fine-tuneado, pero, como el objetivo es medir el rendimiento de los diferentes frameworks, hemos decidido hacer la inferencia con el modelo base por simplicidad, ya que no hay que guardar pesos ni cargarlos; y porque no influye en el rendimiento, ya que con LoRA no se añaden parámetros al modelo que puedan aumentar la latencia. Se ha utilizado el modelo base sin cuantizar para la inferencia.

La evaluación de inferencia de los diferentes frameworks se realizó usando un número fijo de 75 prompts (parámetro modificable en los diferentes notebooks con `SAMPLE_SIZE`). Además, se fijaron una serie de parámetros como la temperatura o el número máximo de tokens, con lo que se consigue aislar el impacto propio de cada framework, evitando introducir variabilidad en los resultados por el contenido de las entradas.

A diferencia del fine-tuning, en la inferencia se introducen métricas adicionales relacionadas con la latencia, el consumo de memoria y la escalabilidad, en las que nos fijaremos para sacar unas conclusiones justas sobre cada framework (se fija la semilla de generación para que los prompts sean siempre los mismos).

Es importante destacar que, debido a las diferencias en la estructura entre los frameworks, la inferencia no se ejecuta de la misma manera en todos. Por ejemplo, un framework se ejecuta de manera secuencial, mientras que otro la ejecuta de manera paralela, lo que da una clara ventaja

en resultados a este segundo caso, dado que puede ejecutar múltiples prompts al mismo tiempo. Pese a esto, se ha decidido ejecutar y evaluar todos los frameworks en su uso natural, es decir, teniendo en cuenta estas diferencias y reflejando escenarios de despliegue realistas.

Los cuatro frameworks evaluados son:

- Transformers: es la librería de Hugging Face, en la que la inferencia se realiza de forma secuencial, procesando cada prompt de manera independiente.
- Unsloth: introduce optimizaciones específicas sobre el flujo de inferencia estándar de Transformers, y está más optimizado para hacer fine-tuning. Se evaluó sin aplicar cuantización, para aislar el impacto del framework.
- vLLM: agrupa múltiples peticiones y las ejecuta de forma paralela, utilizando mecanismos avanzados de gestión de memoria como PagedAttention. Lo que lo hace adecuado para entornos de servidor.
- Ollama: prioriza la simplicidad y la eficiencia. La inferencia se realiza a través de un modelo cliente-servidor, que permite evaluar el rendimiento end-to-end del sistema.

3. Evaluación

3.1. Métricas

En la parte de fine-tuning se emplearon las siguientes métricas para cada framework:

1. Tiempo de entrenamiento
2. Consumo de memoria
3. Benchmarks (BLEU, ROUGE-L, BERTScore)

Estas métricas permiten analizar el equilibrio entre tiempo, eficiencia en memoria y resultados del fine-tuning.

Los benchmarks utilizados comparan la salida generada por el modelo con una respuesta de referencia. Esto se adapta bien al fine-tuning porque ayuda a medir el antes y el después de los ajustes. Estas tres métricas se complementan bien en este caso: BLEU mide coincidencia superficial (n-gramas), ROUGE-L se centra en orden y estructura de la respuesta y BERTScore busca la similitud semántica con embeddings.

En la parte de inferencia, para cada framework se midieron las siguientes métricas

- Latencia media por petición: tiempo total necesario para procesar una solicitud de inferencia.
- Throughput: número de tokens generados por segundo.
- Uso de memoria: RAM y/o VRAM reportada durante la ejecución.

Estas métricas permiten analizar el equilibrio entre rendimiento, eficiencia en memoria y escalabilidad.

3.2. Resultados

En la siguiente tabla podemos ver los resultados de los benchmarks en ambos frameworks, tanto para el modelo base como para el modelo fine-tuneado. Se puede observar en la Tabla 1 que el modelo ajustado en Transformers es ligeramente mejor que el modelo base en Transformers. En cambio, en Unsloth el base tiene mejores métricas que el tuneado. Esto se debe a que se han buscado los hiperparámetros en Transformers y se han replicado en Unsloth para comparar el rendimiento del entrenamiento, mostrado en la Tabla 2 (habría que buscar los hiperparámetros en Unsloth también). Se aprecia claramente que el rendimiento del fine-tuning es mejor en Unsloth, consumiendo un tercio de la memoria que Transformers y tardando menos de la mitad en ajustar.

Métrica	Transformers Base	Transformers Lora	Unsloth Base	Unsloth Lora
BLEU	0.0617	0.0633	0.042	0.0238
ROUGE-L	0.218	0.2468	0.166	0.144
BERTScore	0.559	0.585	0.5	0.4088

Tabla 1. Resultados en benchmarks.

Métrica	Transformers	Unsloth
Memoria Activa	2.1 GB	0.74 GB
Tiempo	700 s	311 s

Tabla 2. Resultados del entrenamiento (fine-tuning).

Para la parte de inferencia, la siguiente tabla resume los resultados obtenidos para los 75 prompts evaluados.

Framework	Avg Latency (s)	Throughput (tok/s)	Memory (GB)
Transformers	2.34	34.38	2.07
Unsloth	2.58	36.86	2.0
vLLM*	0.03*	2785.53*	8.84
Ollama	0.57	255.1	0.78

Tabla 3. Resultados cuantitativos obtenidos para inferencia

* La latencia y el throughput corresponden a lo expuesto en la metodología, que, al hacerse de forma paralela, dan estos resultados tan sorprendentes en estas métricas.

Los resultados en esta parte muestran diferencias claras entre los frameworks evaluados, que pueden explicarse por la filosofía del diseño de cada framework.

Transformers presenta la mayor latencia y el menor throughput. Esto se debe principalmente a que Transformers es una librería cuya arquitectura prioriza la compatibilidad universal y la flexibilidad para la investigación sobre la optimización.

Unsloth logra reducir la latencia media y aumentar ligeramente el throughput sin un gran impacto en la memoria. Aunque su foco principal no es la inferencia (se centra en mejorar el fine-tuning), mejora los resultados a nivel de operaciones matriciales ofreciendo una pequeña optimización sobre la base estándar de Transformers.

Por otro lado, **vLLM** destaca por un throughput mucho mayor que el resto. Este resultado ya se ha comentado anteriormente y se valida con su diseño como motor de servicio de alto rendimiento, que permite la ejecución de manera paralela. Este framework implementa PagedAttention, que ayuda a maximizar la ocupación de la GPU. Sin embargo, esta gran mejora en los resultados y la posibilidad de ejecutar en paralelo se consigue a costa de un consumo de memoria notablemente mayor, debido al mantenimiento de grandes cachés de atención y para poder gestionar múltiples peticiones al mismo tiempo. Cuando se ejecuta con un solo prompt, se ve que consigue unos buenos resultados (mejor que Transformers y Unsloth) aunque consume la misma memoria que cuando ejecuta los 75 prompts, lo cual es demasiada memoria para esta tarea.

Por último, **Ollama**, aunque no es tan rápido como vLLM, supera a Transformers y consume muy poca memoria, lo cual se debe a que está diseñado para uso personal y local. Su diseño (apoyado en herramientas como llama.cpp) está hecho para funcionar eficientemente en ordenadores normales. Por eso, se enfoca en gastar los mínimos recursos posibles, sacrificando la potencia por un uso más simple y accesible.

Para igualar un poco y poder comentar, se ha ejecutado vLLM con un prompt, que ha dado el siguiente resultado.

Framework	Avg Latency (s)	Throughput (tok/s)	Memory (GB)
vLLM	1.49	85.86	8.84

Tabla 4. Resultado cuantitativo obtenido para vLLM con 1 prompt

4. Comparación de frameworks

4.1. Comparación centrada en fine-tuning

Transformers

Ofrece un control muy fino sobre todo el proceso de entrenamiento: definición explícita de capas LoRA, hiperparámetros, estrategias de cuantización y gestión del dispositivo. Esta flexibilidad lo hace especialmente adecuado para investigación y experimentación. Sin embargo, el coste es un mayor consumo de memoria y tiempos de entrenamiento más largos, ya que no incluye optimizaciones específicas para fine-tuning eficiente con modelos grandes cuantizados. Además, la correcta configuración requiere más esfuerzo y experiencia por parte del usuario.

Unsloth

Se apoya en Transformers, pero introduce optimizaciones específicas para fine-tuning con LoRA y cuantización. En la práctica, reduce significativamente el uso de memoria y acelera el entrenamiento, permitiendo fine-tuning de modelos más grandes en GPUs con recursos limitados. A cambio, ofrece menos flexibilidad y control interno, ya que abstrae parte de la configuración. Su dependencia exclusiva de GPU también limita su accesibilidad.

Ambos frameworks son muy similares porque Unsloth se construye directamente sobre Transformers. Mantiene su misma base y compatibilidad, pero la especializa para fine-tuning eficiente, introduciendo optimizaciones internas orientadas a LoRA y cuantización. Esto hace que el flujo de trabajo y las APIs sean prácticamente las mismas, aunque Unsloth sacrifica parte de la flexibilidad de Transformers a cambio de mayor rendimiento y menor consumo de memoria durante el entrenamiento.

4.2. Comparación centrada en inferencia

El análisis de los resultados se centra únicamente en el comportamiento durante el proceso de inferencia y en exponer y explicar los resultados obtenidos. Sin embargo, en esta comparación se pretende ver las diferencias entre los distintos frameworks en el uso dado durante el proyecto.

Transformers ofrece flexibilidad y transparencia, muy útil para la investigación. Ofrece una documentación muy amplia (al pertenecer a Hugging Face y ser una plataforma muy famosa), lo que hace que la configuración y el debugging sean muy accesibles. Sin embargo, como se ha expuesto en los resultados, esta accesibilidad y flexibilidad conlleva que obtenga los peores resultados de los cuatro frameworks. Permite el uso de CPU y GPU, aunque requiere una mejor gestión de la GPU por parte del usuario al no incluir optimizaciones específicas para inferencia ya que el usuario debe indicar el dispositivo usado o los límites de generación.

Unsloth mantiene compatibilidad con Transformers, pero aplica optimizaciones que mejoran el rendimiento en inferencia. En nuestra experiencia, su configuración inicial es algo más delicada, aunque su amplia documentación con notebooks de ejemplo ayuda mucho a saber cómo usarlo. Por otro lado, Unsloth solo se puede usar con GPUs, lo que lo hace inaccesible para personas que disponen únicamente de CPU.

vLLM es el único que tiene un enfoque claro en maximizar el throughput haciendo su ejecución en paralelo. Esto se refleja en los resultados obtenidos, donde destaca cuando hay múltiples peticiones (prompts). A cambio, requiere una mayor cantidad de memoria y presenta una mayor complejidad en la configuración y la depuración. En nuestra experiencia, este framework ha sido más complejo de entender, pero ha dado menos problemas de instalación y ejecución. vLLM solo se puede usar con GPU, aunque hay algunas versiones experimentales para poder usarlo con CPU,

esto puede deberse a que está orientado principalmente a servir modelos ya entrenados en entornos de producción.

Ollama prioriza la simplicidad y la eficiencia en inferencia local. Su configuración es mínima y se pueden ejecutar modelos con un bajo consumo de memoria, especialmente en CPU. Sin embargo, esta facilidad de uso, indica una menor capacidad de control. En la práctica, Ollama es una opción muy adecuada para la inferencia rápida y ligera, aunque es menos adecuado para experimentar.

En resumen, para elegir un framework u otro depende del contexto en el que se quiera usar. Transformers y Unsloth ofrecen más control y son compatibles para investigar, vLLM maximiza el rendimiento para servidores con múltiples peticiones, y Ollama destaca su facilidad y la eficiencia de manera local.

5. Discusión, conclusiones y trabajo futuro

A lo largo de este proyecto, hemos podido adquirir experiencia práctica con fine-tuning e inferencia utilizando Transformers, Unsloth, vLLM y Ollama. Estos son algunos de los frameworks más conocidos y utilizados en entornos profesionales y gracias a la experimentación realizada, tenemos una mejor comprensión del funcionamiento y casos de uso para cada una de estas herramientas.

De cara al fine-tuning, Unsloth es una opción excelente para conseguir un entrenamiento eficiente en sistemas con recursos limitados, siendo aproximadamente 2x más rápido y 3x más barato en términos de memoria que Transformers. Aunque la calidad cayó al utilizar hiperparámetros no optimizados para Unsloth, a pesar de los cambios entre las estructuras, al seguir la misma metodología de fine-tuning se pueden obtener resultados comparables con Transformers manteniendo los beneficios de la optimización de Unsloth.

Transformers se queda como la opción más flexible y transparente, lo que lo hace excelente para investigación y debugging, pero también es la menos competitiva en términos de rendimiento.

Para la inferencia en servidores con muchas peticiones, vLLM es claramente superior por su capacidad de ejecución paralela gracias al mecanismo de PagedAttention. La consecuencia de la baja latencia y alto throughput es el gran consumo de recursos. Se ha visto que, incluso en el caso de inferencia de 1 prompt, el uso de memoria se mantiene alto. Por ello, solo conviene utilizar vLLM si la carga de peticiones justifica los costes computacionales.

Para inferencia local y simple, Ollama consigue una buena latencia y throughput con el menor consumo de memoria de los cuatro frameworks. Desde nuestro punto de vista, Ollama tuvo el despliegue más directo, a cambio de un menor control sobre la optimización del modelo que otros como vLLM. Se intercambian la compatibilidad y el control por una experiencia sencilla y optimizada para los modelos disponibles.

Cada una de estas herramientas tiene un entorno en el que destaca sobre las demás y este proyecto nos ha ayudado a descubrirlos. Unsloth es una buena opción para entornos en los que se ajustan modelos con una sola GPU, en los que tiene que conseguir un modelo funcional relativamente rápido. Esto nos permite probar con muchos conjuntos de hiperparámetros y el ahorro acumulado de recursos nos permite reducir los costes de proyecto. Transformers destaca en entornos de investigación donde hace falta cambiar alguna parte del proceso (tokenización, métricas, callbacks, loss functions personalizadas, gradientes, etc.). La alta compatibilidad de Transformers también permite trabajar con modelos con estructuras poco comunes. Por otro lado, vLLM destaca para entornos como el backend de un chatbot o de una aplicación que tenga muchos usuarios haciendo peticiones constantemente donde el mayor cuello de botella es el throughput (tokens/segundo). Siempre que sea un entorno de alta actividad, vLLM es una gran opción. Ollama es excelente para desarrollo local y offline. Algunos miembros del grupo ya tenían experiencia con este framework en proyectos personales debido a que su eficiencia hace que sea la mejor opción para ordenadores personales con recursos limitados. Al trabajar offline, la privacidad de datos sensibles es más sencilla de garantizar y esto también es un excelente beneficio en entornos profesionales donde mantener información sensible confidencial es una parte esencial del negocio.

Este proyecto tuvo principalmente un carácter didáctico, pero en el futuro se puede continuar con mejoras y ampliaciones. La escala ha sido relativamente pequeña, por lo que con más recursos se podría indagar más en el comportamiento de cada framework. Para empezar, estos experimentos se podrían aplicar a casos de uso concretos y con aplicaciones reales. Dentro de cada caso, analizar el conjunto de herramientas candidatas y analizar cuál se ajusta mejor. Además, la escala de los

modelos y datasets se podría ampliar significativamente para obtener mejores resultados y así observar el comportamiento de los modelos en escalas de parámetros más parecidas a las que se usan en entornos profesionales. De cara a medir los resultados, haría falta ampliar las métricas utilizadas porque BLEU, ROUGE y BERTScore no siempre miden completamente la calidad de las respuestas. En nuestro estudio, observamos este fenómeno y, mediante una evaluación humana inicial, notamos diferencias significativas en algunas respuestas y las métricas utilizadas no lo representaban en la misma escala. Por otro lado, podríamos medir la robustez y capacidad de generalización de los modelos. Finalmente, también podríamos indagar más en el impacto que puede llegar a tener la cuantización con Ollama. Al ser muy eficiente, se podrían utilizar modelos enormes y medir cómo la cuantización afecta a la inferencia, incluida la pérdida de calidad. Este es el punto fuerte de los modelos de Ollama en local, y medir el impacto de la cuantización puede ayudar a encontrar la mejor configuración para un sistema local sencillo.